

# TransparentGS: Fast Inverse Rendering of Transparent Objects with Gaussians

LETIAN HUANG, State Key Lab for Novel Software Technology, Nanjing University, China

DONGWEI YE, State Key Lab for Novel Software Technology, Nanjing University, China

JIALIN DAN and CHENGZHI TAO, State Key Lab for Novel Software Technology, Nanjing University, China

HUIWEN LIU, TMCC, College of Computer Science, Nankai University, China

KUN ZHOU, State Key Lab of CAD & CG, Zhejiang University, China and Institute of Hangzhou Holographic Intelligent Technology, China

BO REN, TMCC, College of Computer Science, Nankai University, China

YUANQI LI and YANWEN GUO, State Key Lab for Novel Software Technology, Nanjing University, China

JIE GUO\*, State Key Lab for Novel Software Technology, Nanjing University, China

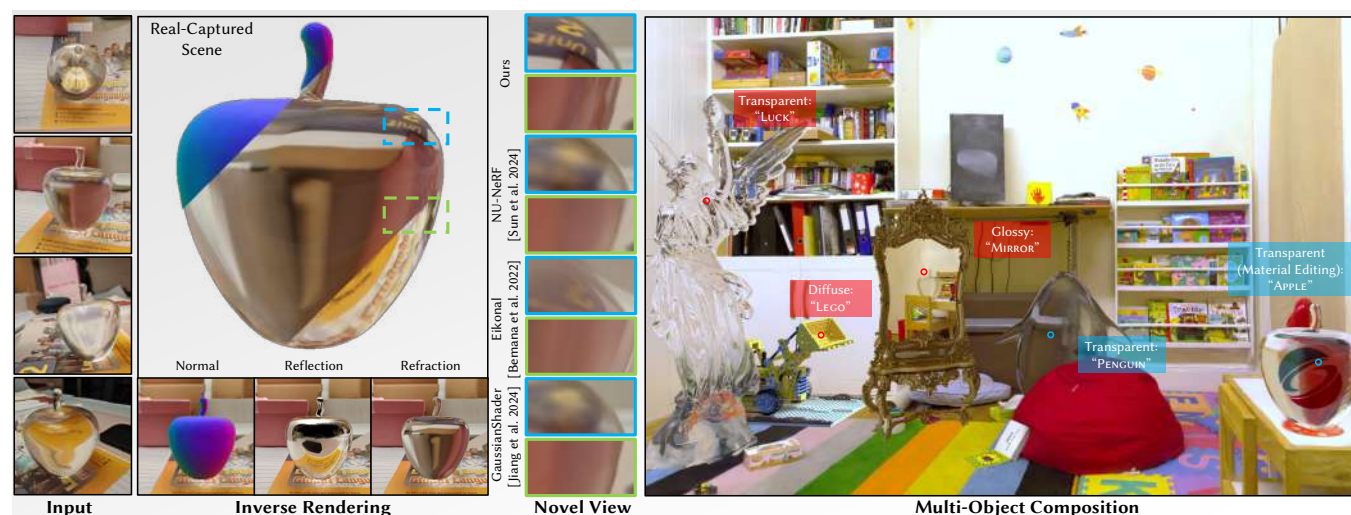


Fig. 1. TransparentGS is a novel inverse rendering pipeline based on 3D-GS, specifically designed for reconstructing transparent objects under various light and viewing conditions. It delivers high-quality reconstruction results within **1 hour**, enabling real-time novel view synthesis and convenient re-rendering of scenes with challenging transparent objects. Compared to state-of-the-art methods, it achieves both high-fidelity refraction (blue box) and reflection (green box) with nearby contents (indirect light), thereby paving the way for complex secondary ray effects in the scene composited of multiple objects.

\* Corresponding authors.

Authors' addresses: Letian Huang, State Key Lab for Novel Software Technology, Nanjing University, Nanjing, China, lthuang@smail.nju.edu.cn; Dongwei Ye, dongweiye@smail.nju.edu.cn, State Key Lab for Novel Software Technology, Nanjing University, Nanjing, China; Jialin Dan, danjialin@smail.nju.edu.cn; Chengzhi Tao, tcz\_tao@smail.nju.edu.cn, State Key Lab for Novel Software Technology, Nanjing University, Nanjing, China; Huiwen Liu, TMCC, College of Computer Science, Nankai University, Tianjin, China, huiwenliu@mail.nankai.edu.cn; Kun Zhou, State Key Lab of CAD & CG, Zhejiang University, Hangzhou, China and Institute of Hangzhou Holographic Intelligent Technology, Hangzhou, China, kunzhou@zju.edu.cn; Bo Ren, TMCC, College of Computer Science, Nankai University, Tianjin, China, rb@nankai.edu.cn; Yuanqi Li, yuanqili@nju.edu.cn; Yanwen Guo, ywguo@nju.edu.cn, State Key Lab for Novel Software Technology, Nanjing University, Nanjing, China; Jie Guo, State Key Lab for Novel Software Technology, Nanjing University, Nanjing, China, guojie@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

The emergence of neural and Gaussian-based radiance field methods has led to considerable advancements in novel view synthesis and 3D object reconstruction. Nonetheless, specular reflection and refraction continue to pose significant challenges due to the instability and incorrect overfitting of radiance fields to high-frequency light variations. Currently, even 3D Gaussian Splatting (3D-GS), as a powerful and efficient tool, falls short in recovering transparent objects with nearby contents due to the existence of apparent secondary ray effects. To address this issue, we propose TransparentGS, a fast inverse rendering pipeline for transparent objects based on 3D-GS. The main contributions are three-fold. Firstly, an efficient representation of transparent objects, transparent Gaussian primitives, is designed to enable specular refraction through a deferred refraction strategy. Secondly, we leverage Gaussian light field probes (GaussProbe) to encode both ambient light and nearby contents in a unified framework. Thirdly, a depth-based

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
0730-0301/2025/8-ART \$15.00  
<https://doi.org/10.1145/3730892>

iterative probes query (IterQuery) algorithm is proposed to reduce the parallax errors in our probe-based framework. Experiments demonstrate the speed and accuracy of our approach in recovering transparent objects from complex environments, as well as several applications in computer graphics and vision.

CCS Concepts: • **Computing methodologies** → **Image-based rendering; Point-based models.**

Additional Key Words and Phrases: 3D gaussian splatting, transparent object, inverse rendering

#### ACM Reference Format:

Letian Huang, Dongwei Ye, Jialin Dan, Chengzhi Tao, Huiwen Liu, Kun Zhou, Bo Ren, Yuanqi Li, Yanwen Guo, and Jie Guo. 2025. TransparentGS: Fast Inverse Rendering of Transparent Objects with Gaussians. *ACM Trans. Graph.* 44, 4 (August 2025), 16 pages. <https://doi.org/10.1145/3730892>

## 1 INTRODUCTION

Reconstructing 3D scenes from multiple 2D images of different views and synthesizing novel views has been a long-standing task in computer graphics and vision. With the advent of deep learning, the trend in solving this task has been spearheaded by Neural Radiance Fields (NeRFs) [Mildenhall et al. 2020] and its variants [Barron et al. 2022; Müller et al. 2022], which achieve photorealistic visual quality by using volume rendering with implicit fields. Recently, as the demand for real-time rendering continues to increase, 3D Gaussian Splatting (3D-GS) [Kerbl et al. 2023] offers a more efficient explicit representation that can achieve real-time rendering by modeling radiance fields as 3D Gaussian primitives.

Unfortunately, until now it remains challenging to reconstruct transparent objects under arbitrary light conditions and enable real-time novel view synthesis. The challenge arises from the frequent appearance variations of transparent objects across different viewpoints, due to the intricate interplay of reflection and refraction as light traverses the material. Consequently, both MLPs with the directional encodings in NeRF and the spherical harmonic functions (SH) in 3D-GS face difficulties in modeling specular reflection and refraction accurately.

Currently, the most effective reconstruction methods for transparent objects are based on implicit neural representations [Bemana et al. 2022; Gao et al. 2023; Li et al. 2020; Sun et al. 2024]. However, these methods are notorious for their significant computational overhead during training, and also do not facilitate real-time rendering, as explained and compared in Tab. 1. 3D-GS-based methods offer high efficiency in 3D reconstruction and novel view synthesis [Wu et al. 2024b], but existing methods have their own limitations. In particular, these methods [Jiang et al. 2024; Ye et al. 2024], originally designed for reflective scenes, cannot be easily applied to transparent objects. The difficulty is exacerbated when the transparent object is surrounded by nearby objects. Due to the rasterization-based formulation, 3D-GS is constrained to ideal pinhole cameras and lacks support for secondary ray effects such as refraction and inter-reflection [Huang et al. 2024; Moenne-Loccoz et al. 2024].

In this paper, we propose *TransparentGS*, a novel inverse rendering framework that builds upon 3D-GS to efficiently manage transparent objects in a variety of light and viewing conditions, supporting both refraction and inter-reflection with nearby contents. In this framework, transparent objects are modeled by *transparent*

Table 1. **Comparison of transparent object reconstruction methods** in terms of training time (A), rendering time (B), and the capability of supporting ambient light (C), nearby contents (indirect light) (D), high-frequency refraction details (E), accurate reflection-refraction decoupling (F), colored refraction (G), and re-rendering (e.g., relighting or material editing [Khan et al. 2006]) (H).

| Methods                      | A                    | B                | C | D | E | F | G | H |
|------------------------------|----------------------|------------------|---|---|---|---|---|---|
| Eikonal [Bemana et al. 2022] | slow (>10h)          | offline          | ✓ | ✓ | × | × | × | × |
| NEMTO [Wang et al. 2023]     | slow (>10h)          | offline          | ✓ | × | ✓ | ✓ | × | × |
| Gao et al. [2023]            | slow (>10h)          | offline          | × | ✓ | ✓ | × | × | ✓ |
| NU-NeRF [Sun et al. 2024]    | slow (>7h)           | offline          | ✓ | ✓ | × | ✓ | × | ✓ |
| <b>TransparentGS (Ours)</b>  | <b>fast (&lt;1h)</b> | <b>real-time</b> | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

*Gaussian primitives*, which explicitly encode both geometric and material properties within 3D Gaussians. These new primitives are convenient to be integrated into a dedicated *physically-based deferred rendering pipeline*, enabling the consolidation of mesh and GS for secondary ray effects. To handle both ambient light and nearby contents in the scene, we introduce *Gaussian light field probes (GaussProbe)* to capture the local light field of each transparent object by placing a sparse set of caches around it. These probes can be baked and updated fastly based on an optimal projection strategy [Huang et al. 2024]. To address the parallax issue inherent to the probes and enhance the details of refraction/inter-reflection, we design a *depth-based iterative probes query algorithm (IterQuery)* which achieves plausible results only after a few iterations.

Compared to previous work, our new framework has several advantages as listed in Tab. 1. In particular, we achieve the first fast inverse rendering with secondary ray effects of transparent objects *within one hour*, enabling real-time novel view synthesis of transparent scenes and complex secondary ray effects in the interplay, as illustrated in Fig. 1.

In summary, the contributions of this work are as follows:

- We design an efficient representation of transparent objects, transparent Gaussian primitives, which enables specular refraction through a deferred refraction strategy.
- We leverage Gaussian light field probes (GaussProbe) to encode both ambient light and nearby contents within a unified framework.
- We propose a depth-based iterative probe query algorithm (IterQuery) to mitigate the parallax errors in our probe-based framework.

## 2 RELATED WORK

In this section, we briefly review the reconstruction and novel view synthesis methods for transparent objects, and also discuss various light representations in inverse rendering.

*Transparent object reconstruction.* To reconstruct transparent objects, a few early methods [Huynh et al. 2010; Ihrke et al. 2010; Kutulakos and Steger 2008; Qian et al. 2016; Shao et al. 2024; Stets et al. 2017; Wetzstein et al. 2011] employ dedicated hardware setups like polarization cameras or require additional information beyond a simple set of images. Subsequent works relax the constraints but still rely on unique patterns to infer the correspondence between

the camera ray and the background [Li et al. 2023; Lyu et al. 2020; Wu et al. 2018; Xu et al. 2022] or require manual annotations [Li et al. 2020; Xu et al. 2022]. Recently, with the emergence of neural radiance fields [Mildenhall et al. 2020] in novel view synthesis tasks, numerous studies leverage neural implicit representations to achieve transparent object reconstruction [Gao et al. 2023; Sun et al. 2024; Tong et al. 2023; Zhan et al. 2023]. Bemana et al. [2022] propose novel view synthesis by learning an IoR field alongside radiance and density, addressing the bent light path in the field without modeling the surface geometry of objects. The eikonal equation deals with refraction and total internal reflection but not separation into partial reflection and refraction, which poses challenges for re-rendering. NEMTO [Wang et al. 2023] employs a neural network to predict refracted directions and performs physically-based shading under the assumption of ambient light. Gao et al. [2023] propose a two-stage method to predict multi-view silhouettes and reconstruct the shape of a refractive object on an opaque plane, which serves as both a geometry and appearance prior. Both methods [Gao et al. 2023; Wang et al. 2023] are limited by their light representations and cannot handle realistic scenes that simultaneously include nearby objects and light. NU-NeRF [Sun et al. 2024] employs a two-stage reconstruction strategy to handle nested transparent objects. However, it utilizes a network to predict refracted light, making it without refraction details. Moreover, these neural rendering methods are very time-consuming due to the intensive network inference and sampling required. Currently, all the aforementioned neural methods require far more time than our method to reconstruct transparent objects.

*Light representation.* When reconstructing specular or transparent objects, it is essential to account for their surrounding environment to accurately analyze the reflected and refracted light paths. Previous methods for novel view synthesis of transparent objects [Wang et al. 2023] and inverse rendering of opaque objects [Jiang et al. 2024; Lai et al. 2025; Liang et al. 2023] typically assume that all light comes from an infinitely faraway environment and approximate it using an environment maps, which neglects secondary ray effects. To model indirect light, TensoIR [Jin et al. 2023], based on a tensor-factorized representation [Chen et al. 2022], explicitly computes ray integrals to ensure accurate visibility and indirect light in radiance field rendering, which is highly time-consuming. NeLLF++ [Zhang et al. 2023] proposes to marry an incident light field and an outgoing radiance field through physically-based rendering, which enables the handling of specular surfaces and inter-reflection, but often results in overly smooth outcomes and requires extensive training time. Gao et al. [2023] assume all refracted light comes from a perfect textured plane, overlooking distant ambient light. Following NeRO [Liu et al. 2023], NU-NeRF [Sun et al. 2024] employs two MLPs with Integrated Directional Encoding (IDE) [Verbin et al. 2022] to model the ambient light and indirect inter-reflected light from nearby objects, respectively. It further utilizes an MLP that simultaneously inputs position and view direction to predict refracted light, leading to over-blurred results. NeLF-Pro [You et al. 2024] models a scene using light field probes in the context of NeRF [Mildenhall et al. 2020]. It primarily models outgoing radiance rather than incident light, which makes it challenging to directly query the incident light during the inverse rendering process. TraM-NeRF [Holland

et al. 2024] traces reflected rays and introduces a radiance estimator that combines volume and reflected radiance integration to model inter-reflection. However, it cannot handle refractive objects and does not support real-time rendering. Liang et al. [2024] and Gao et al. [2024] employ spherical harmonics (SH) to model indirect light for opaque objects, which is not suitable for representing the high-frequency details of specular refraction. Zhou et al. [2024] propose a novel and insightful Gaussian-based offline path tracer, which, however, does not support fast inverse rendering of transparent objects. Our proposed Gaussian light field probes, on the other hand, can model indirect light with high fidelity and performance for specular refraction and inter-reflection.

### 3 METHOD

In this section, we present our method, TransparentGS, tailored for the fast inverse rendering of transparent objects. As depicted in Fig. 2, our TransparentGS consists of two parts. The first is the representation of transparent objects, transparent Gaussian primitives, which encompasses shape attributes following 3D Gaussian Splatting (3D-GS) [Kerbl et al. 2023]. To enhance the representation capability of refraction, we extend the primitives with additional attributes, including normals and parameters of the Bidirectional Scattering Distribution Function (BSDF) [Bartell et al. 1981]. To render detailed specular refraction with the primitives, we employ a deferred refraction method. The second is our proposed Gaussian light field probes (GaussProbe), which not only represent ambient light similar to environment maps but also indirect light from nearby contents. To eliminate parallax issues, we develop a depth-based iterative probes query (IterQuery) algorithm dedicated to Gaussian light field probes.

#### 3.1 Preliminaries

*3D Gaussian Splatting.* 3D Gaussian Splatting (3D-GS) [Kerbl et al. 2023] constructs a scene using 3D Gaussian primitives  $\mathcal{G}(\cdot)$ , each characterized by position  $\mu$ , a covariance matrix  $\Sigma$ , opacity  $\alpha$ , and spherical harmonics coefficients  $\text{SH}(\cdot)$  representing view-dependent appearance. To render an image, the 3D Gaussian is transformed into the camera space with world-to-camera transform matrix  $\mathbf{W}$  and projected to 2D image space through a local affine approximation Jacobian matrix  $\mathbf{J}$  [Zwicker et al. 2002]:

$$\Sigma_{2D} = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^T\mathbf{J}^T \quad (1)$$

where  $\Sigma_{2D}$  is the covariance matrix of the projected Gaussian  $\mathcal{G}^{2D}$ . Next, 3D-GS [Kerbl et al. 2023] employs volumetric  $\alpha$ -blending with weights  $\alpha = \alpha\mathcal{G}^{2D}(\cdot)$  to integrate alpha-weighted appearance. The formulation is similar to NeRF [Mildenhall et al. 2020]:

$$C = \sum_{i=1}^N T_i \alpha_i \text{SH}_i(\mathbf{v}) \quad (2)$$

where  $N$  is the number of primitives within a pixel, and  $i$  is the index of the primitives.  $\mathbf{v}$  denotes the view direction, and  $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$  is the accumulated transmittance.

*Rendering equation.* To achieve physically photorealistic rendering of surface geometry, we follow the rendering equation [Kajiya

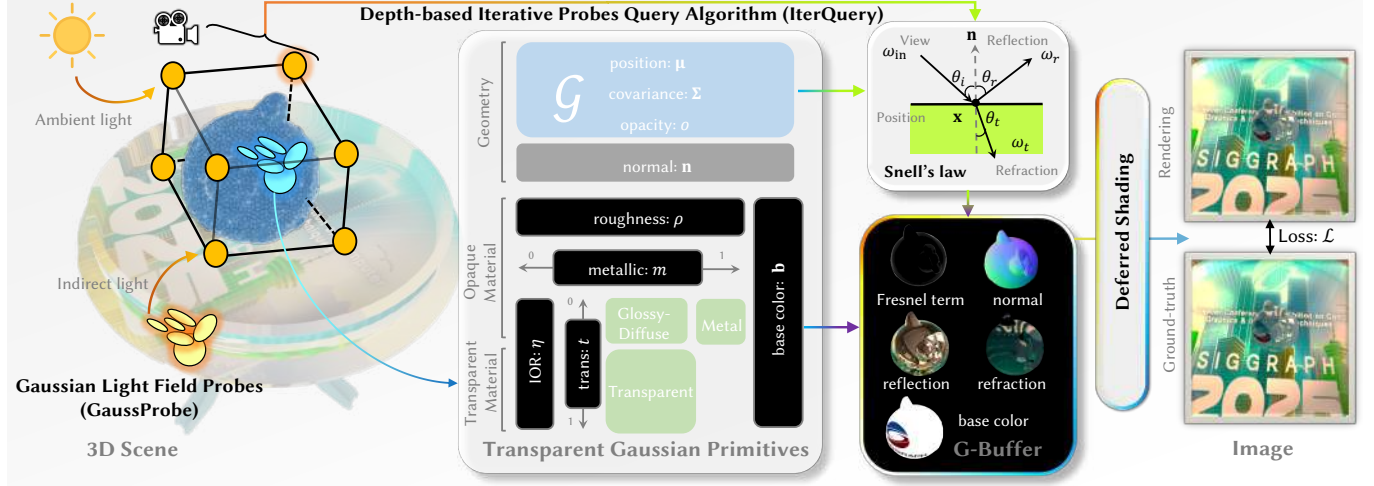


Fig. 2. **The overview of our TransparentGS pipeline.** Each 3D scene is firstly separated into transparent objects and opaque environment using SAM2 [Ravi et al. 2024] guided by GroundingDINO [Liu et al. 2024]. For transparent objects, we propose transparent Gaussian primitives, which explicitly encode both geometric and material properties within 3D Gaussians. And the properties are rasterized into maps for subsequent deferred shading. For the opaque environment, we recover it with the original 3D-GS, and bake it into GaussProbe surrounding the transparent object. The GaussProbe are then queried through our IterQuery algorithm to compute reflection and refraction.

1986] accounting for both reflection and transmission, and employ the bidirectional scattering distribution function (BSDF):

$$L = \int_{\Omega} L_{in}(\mathbf{x}, \omega_{in}) f(\mathbf{x}, \omega, \omega_{in}) |\omega_{in} \cdot \mathbf{n}| d\omega_{in}. \quad (3)$$

$L$  is the shading result,  $L_{in}$  is the incident radiance,  $\mathbf{n}$  is the surface normal, and  $\omega, \omega_{in}$  are the outgoing and incident directions.  $f$  denotes the BSDF at surface point  $\mathbf{x}$  which can be further subdivided as the bidirectional reflectance distribution function (BRDF) and the bidirectional transmittance distribution function (BTDF).

### 3.2 Transparent Gaussian Primitives

The spherical harmonics in 3D-GS [Kerbl et al. 2023] struggle to accurately capture the high-frequency variations of appearance in specular reflection and refraction, leading to either overfitting, which causes degradation in novel views, or highly blurred results. To address this, we propose a novel representation, transparent Gaussian primitives, that involves the physically-based rendering pipeline rather than computing spherical harmonics directly.

**Parameterization.** As shown in Fig. 2, we parameterize the surface geometry as normal  $\mathbf{n}$  and the surface material as roughness  $\rho$  and metallic  $m$ . For transparent objects, it is essential to account for transmission and refraction at the surface. Thus, we utilize two additional material parameters: transparency  $t$  and the index of refraction (IOR)  $\eta$ . The transparency  $t$  is used to interpolate between the opaque and transparent material. Additionally, to preserve the high running performance, we retain the shape parameters of the original 3D-GS [Kerbl et al. 2023] for  $\alpha$ -blending and rasterization.

The opacity attribute plays a key role in 3D-GS [Kerbl et al. 2023] even for opaque objects, as 3D-GS relies on volumetric  $\alpha$ -blending. For transparent objects, merely using a lower opacity to represent transparency will fail to represent the surface, which in turn makes it

hard to compute refraction and reflection. Therefore, we introduce an additional parameter  $t$  to facilitate applications in computer graphics, such as material editing [Khan et al. 2006].

**Surface reflection and refraction.** In this paper, we explicitly represent surface reflection and refraction through two separate BSDFs:

$$f = (1 - t)f_r + tf_t \quad (4)$$

where  $f_r$  denotes BRDF and  $f_t$  denotes BTDF. For opaque materials with  $t = 0$ , we follow the Cook-Torrance model [1981]. And for the reflective component of metals or transparent objects with  $\rho = 0$ , we treat it as perfect specular reflection:

$$f_r = F \frac{\delta(\omega - \omega_r)}{|\omega_{in} \cdot \mathbf{n}|}, \quad \text{with } \omega_r = 2(\omega_{in} \cdot \mathbf{n})\mathbf{n} - \omega_{in} \quad (5)$$

where  $\omega_r$  denotes the analytical reflected direction,  $\delta$  denotes the Dirac delta function, and Fresnel term  $F$  can be approximated using Schlick Approximation [Schlick 1994]. Similarly, for the transmissive component of transparent objects with  $\rho = 0$ , we consider only the perfect specular refraction:

$$f_t = (1 - F) \frac{\delta(\omega - \omega_t)}{|\omega_{in} \cdot \mathbf{n}|} \quad (6)$$

where  $\omega_t$  refers to the refracted direction. As depicted in Fig. 2, the analytical refracted direction is obtained according to Snell's law.

**Deferred refraction.** A critical aspect of employing 3D-GS for inverse rendering is the shading scheme. Generally, the shading strategy can be categorized into two types: forward shading [Dihlmann et al. 2024; Gao et al. 2024] and deferred shading [Deering et al. 1988; Wu et al. 2024a; Ye et al. 2024]. “Forward” refers to performing shading first, followed by  $\alpha$ -blending, while “deferred” does the opposite. Prior studies [Lai et al. 2025; Ye et al. 2024] have highlighted the impact of operation order on specular reflection. Inspired by

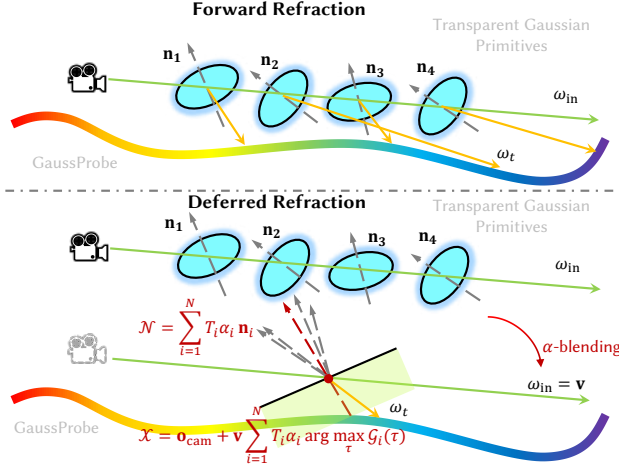


Fig. 3. **Difference between forward and deferred refraction strategies.** The gray arrows denote the normal attributes  $\mathbf{n}_i$  of transparent Gaussian primitives (the blue ellipsoids). Deferred refraction integrates the alpha-weighted normal  $\mathcal{N}$  and the alpha-weighted hitting point  $\mathcal{X}$ , and samples the GaussProbe with the single refracted ray  $\omega_t$  (the orange arrow).

some work on deferred reflection [Wu et al. 2024a; Ye et al. 2024], we explore the possibility of adopting deferred shading in handling specular refraction. Specifically, we derive the transmissive color using Eqn. (3) and Eqn. (6):

$$L_t = \int_{\Omega} L_{in}(\mathbf{x}, \omega_{in}) (1 - F) \delta(\omega - \omega_t) d\omega_{in} \quad (7)$$

$$= (1 - F) L_{in}(\mathbf{x}, \omega_t).$$

Since the  $\alpha$ -blending is a linear operation and Eqn. (7) is not linear with respect to the normal  $\mathbf{n}$ , the operation order has a significant impact on the refraction. Forward refraction averages the shading results of Gaussians. Compared to forward refraction, deferred shading is more adept at capturing transparent objects with specular refraction for its single sampling of the illumination, as shown in Fig. 3. In our deferred refraction strategy (see Fig. 2), we employ a point-based  $\alpha$ -blending approach, similar to Eqn (2), to aggregate all reflection- and refraction-related attributes carried by the primitives. For instance, the alpha-weighted normal is computed by:

$$\mathcal{N} = \sum_{i=1}^N T_i \alpha_i \mathbf{n}_i. \quad (8)$$

Moreover, compared to reflection based on ambient light, we also need to integrate the ray's hitting points for refraction and reflection. However, simply integrating the centers of the primitives  $\boldsymbol{\mu}$  neglects their anisotropy, leading to errors. We adopt a more accurate strategy to compute the hitting point. Considering the primary ray  $\mathbf{o}_{cam} + \tau \mathbf{v}$  with the origin  $\mathbf{o}_{cam}$  and the view direction  $\mathbf{v} = \omega_{in}$ , the Gaussian value on the ray can be represented as a function of the distance  $\tau$ :

$$\mathcal{G}(\tau) = \exp\left(-\frac{1}{2} (\mathbf{o}_{cam} + \tau \mathbf{v} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{o}_{cam} + \tau \mathbf{v} - \boldsymbol{\mu})\right). \quad (9)$$

We analytically compute the maximum value of this function and aggregate the responses of the Gaussians along the ray as follows:

$$\mathcal{X} = \mathbf{o}_{cam} + \mathbf{v} \sum_{i=1}^N T_i \alpha_i \arg \max_{\tau} \mathcal{G}_i(\tau), \quad (10)$$

$$\arg \max_{\tau} \mathcal{G}_i(\tau) = \frac{(\boldsymbol{\mu}_i - \mathbf{o}_{cam})^T \Sigma^{-1} \mathbf{v}}{\mathbf{v}^T \Sigma^{-1} \mathbf{v}}$$

where  $\mathcal{X}$  is the alpha-weighted hitting point and  $\mathcal{G}_i$  denotes the  $i$ -th primitive along the ray. Eqn. (10) considers the anisotropy of transparent Gaussian primitives, where different pixels correspond to different hitting points. The refracted rays  $\omega_t$  and reflected rays  $\omega_r$  are computed using these aggregated attributes stored in the geometry buffer (G-buffer) for subsequent queries.

**Handling Colored Transparent Objects.** Our method can be easily extended to handle absorption in colored transparent objects. We only need to modify Eqn (7) by:

$$L_t = (1 - F) L_{in}(\mathbf{x}, \omega_t) e^{-\sigma(\lambda) d} \quad (11)$$

where  $\sigma(\cdot)$  is the absorption coefficient of the material,  $\lambda$  is the wavelength of the light, and  $d$  denotes the path length that the light travels through the material. We approximate the exponential term (transmittance) using the alpha-weighted base color  $\sum_{i=1}^N T_i \alpha_i \mathbf{b}_i$ , where  $\mathbf{b}_i$  is an optimizable parameter encoded in each transparent Gaussian primitive. Thanks to this design, our method effectively decouples the refraction term from the transparent object's inherent color, which is a challenge that other methods struggle to handle.

### 3.3 Gaussian Light Field Probes

The choice of incident light representation significantly affects inverse rendering. Using an environment map with only two degrees of freedom to represent indirect light from nearby contents causes severe parallax issues. To overcome the challenge of incoherent rays that vanilla 3D-GS [Kerbl et al. 2023] fails to handle, we propose a novel light representation: Gaussian light field probes (GaussProbe). The important symbols in this section are demonstrated in Tab. 2.

**Baking Gaussian light field probes.** Before reconstructing transparent objects, we first bake the Gaussian light field probes. As shown in Fig. 4, we first reconstruct the environment via the vanilla 3D-GS [Kerbl et al. 2023]. We then voxelize the scene and place probes within the voxels surrounding the bounding box of the transparent object. For each probe, we follow the optimal projection [Huang et al. 2024] to render a panoramic image under the setting of a virtual camera. Specifically, we project the Gaussian primitives onto the corresponding tangent planes of the unit sphere with the projection function  $\varphi$  instead of the image plane. The corresponding local affine approximation Jacobian matrix is modified by:

$$\mathbf{J} = \frac{1}{(\mu_x^2 + \mu_y^2 + \mu_z^2)^{\frac{3}{2}}} \begin{bmatrix} \mu_y^2 + \mu_z^2 & -\mu_x \mu_y & -\mu_x \mu_z \\ -\mu_x \mu_y & \mu_x^2 + \mu_z^2 & -\mu_y \mu_z \\ -\mu_x \mu_z & -\mu_y \mu_z & \mu_x^2 + \mu_y^2 \end{bmatrix} \quad (12)$$

where  $\boldsymbol{\mu}' = [\mu_x, \mu_y, \mu_z]^T$  denotes the position of 3D Gaussian in the camera space. Subsequently, for a pixel  $(u, v)$  on the image, we

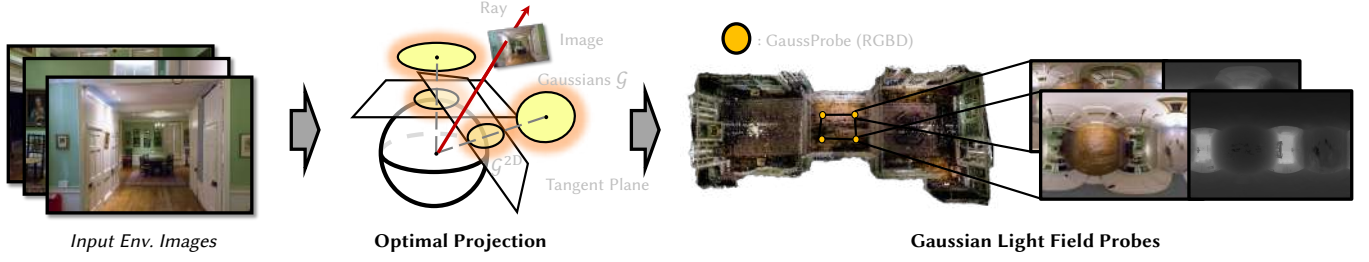


Fig. 4. **Illustration of our baking pipeline for Gaussian light field probes.** Given a set of environmental images with the transparent object removed, we can reconstruct the 3D scene using the original 3D-GS [Kerbl et al. 2023]. We voxelize the scene and place virtual cameras around the bounding box of the transparent object. For each virtual camera, we project the Gaussian primitives onto the tangent plane of the unit sphere, generating tangent-plane Gaussians. Finally, an  $\alpha$ -blending pass bakes the 360° panoramic color and depth maps at each point, which are subsequently stored in the voxels.

Table 2. **List of important notations in GaussProbe and IterQuery.**

| Symbol                           | Definition                                                      |
|----------------------------------|-----------------------------------------------------------------|
| $\mu', \mu_x, \mu_y, \mu_z$      | Position of 3D Gaussians in the camera space                    |
| $\varphi$                        | Optimal projection function                                     |
| $K, i$                           | Number and index of Gaussian light field probes                 |
| $\Phi, \Theta$                   | Color and depth panorama of GaussProbe                          |
| $\mathbf{p}_i, \mathbf{d}_i$     | Position and direction of $i$ -th GaussProbe                    |
| $\mathbf{c}_i, t_i$              | Color and depth of $i$ -th GaussProbe queried by $\mathbf{d}_i$ |
| $\mathbf{o} + t\mathbf{d}$       | Queried ray (e.g., refracted or reflected ray)                  |
| $\mathbf{o} + \hat{t}\mathbf{d}$ | Estimated intersection of the queried ray with Env.             |
| $w_i$                            | Trilinear interpolation weight of $i$ -th GaussProbe            |

cast a ray and compute the intersection with the tangent plane:

$$\mathbf{x}_{2D} = \varphi \left( \begin{pmatrix} \sin \left( \frac{\pi(-W+2u)}{W} \right) \cos \left( \frac{\pi(-H/2+v)}{H} \right) \\ \sin \left( \frac{\pi(-H/2+v)}{H} \right) \\ \cos \left( \frac{\pi(-H/2+v)}{H} \right) \cos \left( \frac{\pi(-W+2u)}{W} \right) \end{pmatrix} \right) \quad (13)$$

where  $H$  and  $W$  denote the resolution of the panorama. During  $\alpha$ -blending, we replace  $\mathcal{G}^{2D}$  in Eqn. (2) with the value of the tangent-plane Gaussian at  $\mathbf{x}_{2D}$ . Additionally, to render the depth of the panorama, we use the Euclidean distance from the primitive to the camera center in place of  $\mu_z$ . Finally, the  $i$ -th probe at position  $\mathbf{p}_i$  stores a color panorama  $\Phi$  and a depth panorama  $\Theta$ , enabling color  $\mathbf{c}_i$  and depth  $t_i$  queries via the direction  $\mathbf{d}_i$  respectively:

$$\mathbf{c}_i, t_i = \Phi(\mathbf{p}_i, \mathbf{d}_i), \Theta(\mathbf{p}_i, \mathbf{d}_i). \quad (14)$$

**Depth-based iterative probes query.** When computing refracted or reflected light during inverse rendering, we should query probes with respect to a given ray. Let  $\mathbf{o}$  and  $\mathbf{d}$  denote the origin and direction of the queried ray, respectively. We also denote  $\mathbf{p}_i$  and  $\mathbf{d}_i$  as the position and direction of the  $i$ -th probe. Fig. 5(d) illustrates that naively querying  $K$  probes along the same direction  $\mathbf{d}_i = \mathbf{d}$  and averaging the results leads to over-blurriness due to the parallax among the probes. To better capture details in specular reflection and refraction, we propose a depth-based iterative probes query (IterQuery) algorithm tailored for our baked GaussProbe. As depicted in Fig. 5, we initialize the directions of probes  $\mathbf{d}_i$  with the

direction of the queried ray  $\mathbf{d}$ :

$$\mathbf{d}_i := \mathbf{d}. \quad (15)$$

Then we query the depth  $t_i$  to obtain the intersection points  $\mathbf{p}_i + t_i \mathbf{d}_i$  of the  $i$ -th probe with the environment:

$$t_i = \Theta(\mathbf{p}_i, \mathbf{d}_i). \quad (16)$$

The trilinear interpolation of the depth maps projected onto the queried ray is given by:

$$\hat{t} = \sum_{i=1}^K w_i ((\mathbf{p}_i + t_i \mathbf{d}_i - \mathbf{o}) \cdot \mathbf{d}) \quad (17)$$

where  $\cdot$  denotes the dot product of vectors and  $w_i$  is the interpolation weights. Then  $\mathbf{d}_i$  are updated using the following formulation:

$$\mathbf{d}_i := \frac{\mathbf{o} + \hat{t}\mathbf{d} - \mathbf{p}_i}{\|\mathbf{o} + \hat{t}\mathbf{d} - \mathbf{p}_i\|}. \quad (18)$$

Then, the updated  $\mathbf{d}_i$  are used to query in the same manner as the previous iteration. The depth queried from the probe (green dashed lines) corresponds to a point on the surface of the 3D scene, while  $\hat{t}$  corresponds to a point on the ray to be queried (gray dashed lines). Therefore, when  $\hat{t}$  remains constant, the ray to be queried should intersect with the scene. We provide a simplified formal proof for the case where  $K = 1$ . Since the convergence condition requires  $\hat{t}$  to remain constant, we can simultaneously solve Eqn. (16), Eqn. (17) and Eqn. (18) to derive the following formulation:

$$\hat{t} = \left( \mathbf{p}_i + \frac{\mathbf{o} + \hat{t}\mathbf{d} - \mathbf{p}_i}{\|\mathbf{o} + \hat{t}\mathbf{d} - \mathbf{p}_i\|} \Theta(\mathbf{p}_i, \mathbf{d}_i) - \mathbf{o} \right) \cdot \mathbf{d} \quad (19)$$

$$\Rightarrow \Theta(\mathbf{p}_i, \mathbf{d}_i) = t_i = \|\mathbf{o} + \hat{t}\mathbf{d} - \mathbf{p}_i\| \quad (20)$$

$$\Rightarrow \mathbf{o} + \hat{t}\mathbf{d} = \mathbf{p}_i + t_i \mathbf{d}_i. \quad (21)$$

After the iteration stops, the final color can be obtained by replacing  $\Theta$  with  $\Phi$  in Eqn. (17). The pseudocode describing this algorithm is provided in List. 1.

Theoretically, the time complexity of the IterQuery algorithm is  $O(TKQ)$ .  $T$ ,  $K$ , and  $Q$  denote the total number of iterations, the number of probes, and the number of queried rays, respectively. The required number of iterations is influenced by the number of probes  $K$  and the depth panorama  $\Theta$ , which we discuss further in the experimental section. Since only a few iterations are needed to

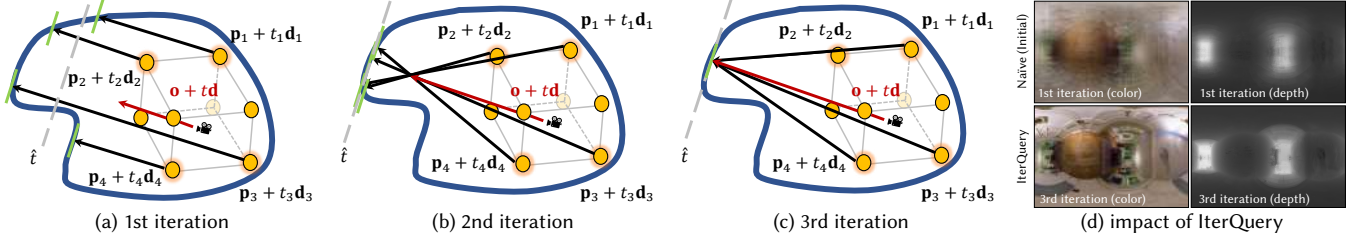


Fig. 5. **Illustration of our depth-based iterative Gaussian probes query (IterQuery) algorithm.** For clarity, we illustrate four selected probes, with positions  $p_1, p_2, p_3$ , and  $p_4$ , chosen from the eight (orange circles). The black camera with the red arrow represents the queried ray (refracted or reflected ray), expressed as  $o + t\mathbf{d}$ . The goal of the algorithm is to determine the correct  $\hat{t}$ , such that  $o + \hat{t}\mathbf{d}$  corresponds to the first intersection between the queried ray and the scene, as well as the correct directions  $\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_4$ .

```

1 def iter_query(probes, ray, total_iter):
2     o, d = ray[... :3], ray[... :3]
3     # calculate the parallax relative to the virtual camera
4     delta_depth = dot(o - probes.pos, d)
5     d_i = d # initialize the directions of probes, Eq. 15
6     while total_iter > 0:
7         # Eq. 16 and the green lines in Fig. 5
8         depth = probes.depth_map.texture(d_i)
9         cos_depth = depth * dot(d, d_i) + delta_depth
10        # Eq. 17 and the gray dashed lines in Fig. 5
11        t_hat = tri_lerp(cos_depth, o, probes.pos)
12        d_i = normalize(o + t_hat * d - probes.pos) # Eq. 18
13        if abs(depth - probes.depth_map.texture(d_i)) <= ε:
14            break # exit if converged in a certain region
15        total_iter -= 1
16        color = probes.color_map.texture(d_i)
17        color = tri_lerp(color, o, probes.pos)
18    return color

```

Listing 1: Pseudocode for our IterQuery algorithm.

achieve a significant improvement compared to the 1st iteration, our approach is both efficient and high-quality.

### 3.4 Multi-Stage Reconstruction

Similar to previous works [Liang et al. 2024; Liu et al. 2023], we adopt the multi-stage reconstruction strategy to achieve the inverse rendering. The first stage involves environment reconstruction and baking, where the baked GaussProbe are used for light computations during geometry and material reconstruction. In the second stage, we leverage transparent Gaussian primitives to reconstruct geometry and material in a physically-based deferred rendering pipeline. During the process, we unify the explicit mesh and GS to trace secondary rays, effectively refining inverse rendering.

**Optimization and losses.** During the process of the multi-stage reconstruction, we impose some regularization terms on the final loss. For the normal regularization term, we follow the previous works [Jiang et al. 2024; Wang et al. 2023]:

$$\mathcal{L}_{\text{normal}} = 1 - \mathcal{N} \cdot \hat{\mathcal{N}}_{\mathcal{D}} \quad (22)$$

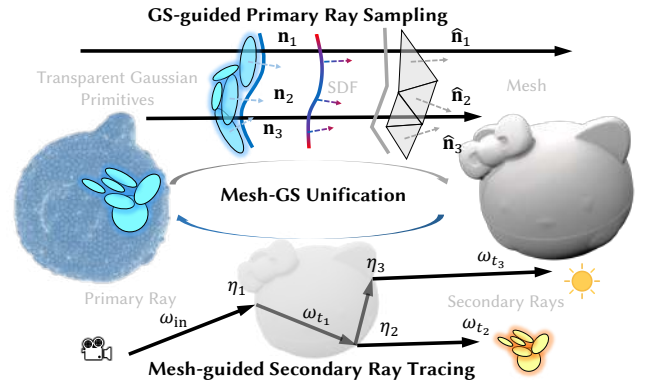


Fig. 6. **Illustration of our mesh-GS fusion strategy.** **GS→Mesh:** We use transparent Gaussian primitives to guide the primary ray sampling of the SDF, efficiently generating an accurate mesh. **Mesh→GS:** We employ the mesh as a proxy for fast secondary ray tracing, which are then utilized in the IterQuery.

where  $\mathcal{N}$  denotes the rendered normal map, and  $\hat{\mathcal{N}}_{\mathcal{D}}$  is the gradient of the rendered depth map. We also incorporate the mask term and the D-SSIM term into the total loss  $\mathcal{L}$ , i.e.,

$$\mathcal{L} = (1 - \lambda_1) \mathcal{L}_1 + \lambda_1 \mathcal{L}_{\text{D-SSIM}} + \lambda_2 \mathcal{L}_{\text{normal}} + \lambda_3 \mathcal{L}_{\text{mask}} \quad (23)$$

where  $\mathcal{L}_1$  and  $\mathcal{L}_{\text{D-SSIM}}$  are the rendering losses.

**Unifying mesh and GS for secondary ray effects.** Physically-based refraction produces at least two bounces of rays inside transparent objects. The rasterization-based 3D-GS [Kerbl et al. 2023], however, is unable to handle secondary rays. Moreover, the second-bounce ray marching is known to be expensive for previous NeRF-based methods [Gao et al. 2023; Li et al. 2024]. While using an MLP to predict the refracted direction [Wang et al. 2023] or light [Sun et al. 2024] is effective, it tends to lose details. Inspired by the joint optimization of GS and SDF [Wu et al. 2024a; Yu et al. 2024], as well as the unification of SDF and mesh [Li et al. 2024], we unify the transparent Gaussian primitives and mesh to address this issue. As detailed in Fig. 6, during the reconstruction, we leverage the hitting points map  $\mathcal{X}$  rendered by transparent Gaussian primitives as guidance for the primary ray sampling of the SDF. The rendered normal map  $\mathcal{N}$  is used to regularize the gradients, which improves the accuracy of



Fig. 7. **3D scene segmentation results on the GLASS scene.** Top left: Image segmentation results. Bottom left: Segmented scene represented by the original 3D-GS [Kerbl et al. 2023]. Right: Probes baked from the segmented scene.

the estimated SDF. We extract the explicit mesh from the SDF field via a marching cube algorithm [Lorensen and Cline 1987]. On the other hand, we employ the extracted mesh as a proxy to efficiently trace secondary rays for subsequent GaussProbe queries. Our mesh-GS unification is somewhat similar to SuGaR [Guédon and Lepetit 2024], which jointly optimizes the mesh and 3D Gaussians located on the surface of the mesh. However, our method focuses more on leveraging the fast reconstruction capability of 3D-GS [Kerbl et al. 2023] and the efficiency of mesh in tracing secondary rays to further refine inverse rendering.

## 4 EXPERIMENTS

### 4.1 Experiment Settings

**Implementation details.** We implement our method based on the PyTorch framework in 3D-GS [Kerbl et al. 2023] with the Adam optimizer [Kingma 2014]. Currently, for the losses, we set  $\lambda_1 = 0.2$ ,  $\lambda_2 = 0.2$ ,  $\lambda_3 = 1$  in all the tests. We employ the pre-trained SAM2 [Kirillov et al. 2023; Ravi et al. 2024] guided by GroundingDINO [Liu et al. 2024] for the 3D scene segmentation, as shown in Fig. 7. Specifically, by entering text prompts, in conjunction with RGB images, GroundingDINO can generate bounding boxes for the transparent object. Subsequently, these bounding boxes can be utilized as box prompts for SAM2 to produce segmentation results. These results act as regularization for Gaussians, facilitating 3D scene segmentation. For our Gaussian light field probes, we set the iteration count of the IterQuery algorithm to 5 and empirically set the number of Gaussian probes to 8 or 64. For all baseline methods, we adopt their official implementations.

**Datasets.** We conduct evaluation on several datasets with transparent objects, including synthetic datasets and real-captured datasets. Specifically, we take one scene including a refractive object with unknown geometry from Bemana et al. [2022]: GLASS, and capture 6 scenes by ourselves: HALFBALL, APPLE, DOLPHIN, PENGUIN, MOUSE and BIRD. We use a smartphone camera to capture 80 ~ 200 views

Table 3. **Quantitative comparison of novel view synthesis results on the real-captured datasets with colorless transparent objects.**

| Methods | GLASS        |              |              | HALFBALL     |              |              | APPLE        |              |              |
|---------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|         | PSNR↑        | SSIM↑        | LPIPS↓       | PSNR↑        | SSIM↑        | LPIPS↓       | PSNR↑        | SSIM↑        | LPIPS↓       |
| Eikonal | <b>27.15</b> | 0.941        | 0.057        | 27.43        | 0.890        | 0.144        | 20.47        | 0.951        | 0.067        |
| GShader | 26.52        | 0.951        | 0.052        | 27.29        | 0.953        | 0.096        | 20.97        | 0.955        | 0.068        |
| NU-NeRF | 26.78        | 0.942        | 0.071        | 27.48        | 0.946        | 0.149        | 22.25        | 0.963        | 0.057        |
| Ours    | 27.12        | <b>0.952</b> | <b>0.044</b> | <b>28.07</b> | <b>0.954</b> | <b>0.084</b> | <b>23.05</b> | <b>0.965</b> | <b>0.047</b> |

Table 4. **Quantitative comparison of novel view synthesis results on the real-captured datasets with colored transparent objects.**

| Scenes  | Ours         |              |              | GShader      |              |        | NU-NeRF      |              |        |
|---------|--------------|--------------|--------------|--------------|--------------|--------|--------------|--------------|--------|
|         | PSNR↑        | SSIM↑        | LPIPS↓       | PSNR↑        | SSIM↑        | LPIPS↓ | PSNR↑        | SSIM↑        | LPIPS↓ |
| PENGUIN | 22.09        | <b>0.832</b> | <b>0.255</b> | 21.51        | 0.806        | 0.271  | <b>22.29</b> | 0.821        | 0.318  |
| DOLPHIN | 22.66        | 0.832        | <b>0.142</b> | 22.60        | 0.842        | 0.158  | <b>22.70</b> | <b>0.833</b> | 0.306  |
| MOUSE   | <b>20.37</b> | <b>0.695</b> | <b>0.154</b> | 19.53        | 0.686        | 0.211  | 19.86        | 0.673        | 0.300  |
| BIRD    | 21.09        | 0.830        | <b>0.136</b> | <b>21.30</b> | <b>0.832</b> | 0.161  | 21.00        | 0.820        | 0.306  |
| Average | <b>21.55</b> | <b>0.797</b> | <b>0.172</b> | 21.24        | 0.792        | 0.200  | 21.46        | 0.787        | 0.308  |

Table 5. **Quantitative comparison of novel view synthesis and inverse rendering results on the synthetic dataset.** The refraction term of GShader [Jiang et al. 2024] is marked as “N/A” due to its incapability of modeling refracted light.

| Methods | Novel View Synthesis |              |              |             | Normal       | Reflection   | Refraction | Base Color   |
|---------|----------------------|--------------|--------------|-------------|--------------|--------------|------------|--------------|
|         | PSNR↑                | SSIM↑        | LPIPS↓       | MAE↓        | PSNR↑        | PSNR↑        | PSNR↑      | PSNR↑        |
| GShader | 24.05                | 0.922        | 0.069        | 26.51       | 12.44        | N/A          |            | 13.19        |
| NU-NeRF | 22.52                | 0.759        | 0.266        | 16.02       | 13.65        | 19.90        |            | 17.08        |
| Ours    | <b>25.66</b>         | <b>0.935</b> | <b>0.064</b> | <b>5.53</b> | <b>17.60</b> | <b>22.87</b> |            | <b>21.51</b> |

for each scene. To demonstrate that our method can handle indirect light from nearby contents beyond opaque planes [Gao et al. 2023], we place several geometrically and texturally complex objects around the transparent objects. Additionally, the last 4 scenes are designed to validate our modeling of colored transparent objects. To qualitatively compare the decoupling and reconstruction quality of our method with other approaches, we render a synthetic dataset with ground truth maps using the Blender: KITTY, DOG and CAT.

**Baselines and metrics.** We compare our method against the following baselines:

- **GShader** [Jiang et al. 2024]: an inverse rendering framework for glossy objects based on 3D-GS [Kerbl et al. 2023].
- **Eikonal** [Bemana et al. 2022]: the state-of-the-art (SOTA) novel view synthesis method for refractive objects based on NeRF [Mildenhall et al. 2020].
- **NU-NeRF** [Sun et al. 2024]: the SOTA reconstruction method for transparent objects based on SDF [Ge et al. 2023; Wang et al. 2021].

We do not compare with NEMTO [Wang et al. 2023] and Gao et al. [2023], as they do not work with complex nearby objects and NU-NeRF has already been compared with both. For the novel view and the decoupled reflection, refraction, and base color components,

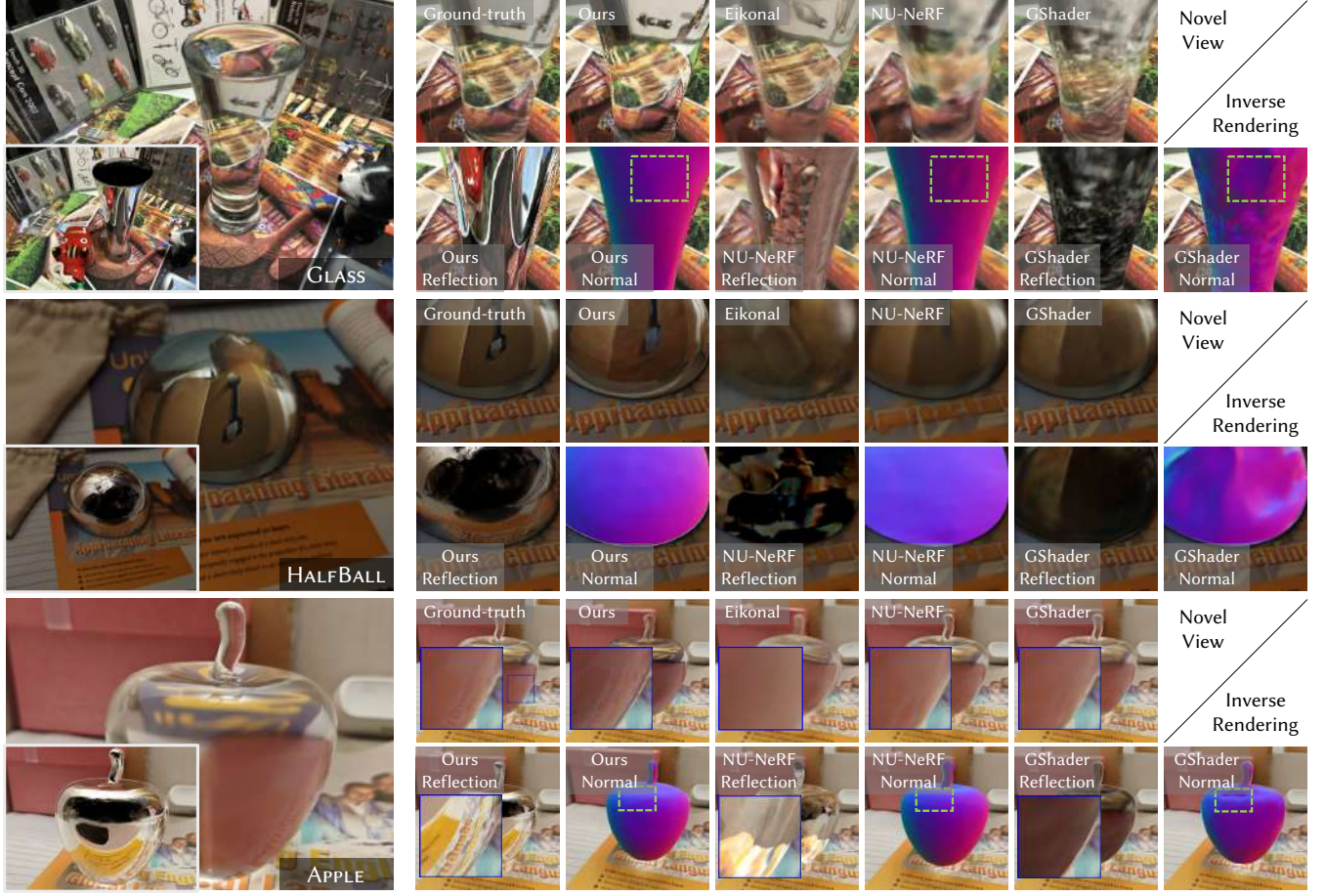


Fig. 8. **Qualitative comparison of novel-view synthesis and inverse rendering results on real-captured scenes with colorless transparent objects.** The left image showcases the transparent object along with its specular reflection component decoupled using our method. The black regions in the reflection component represent the areas that are not visible in the dataset.

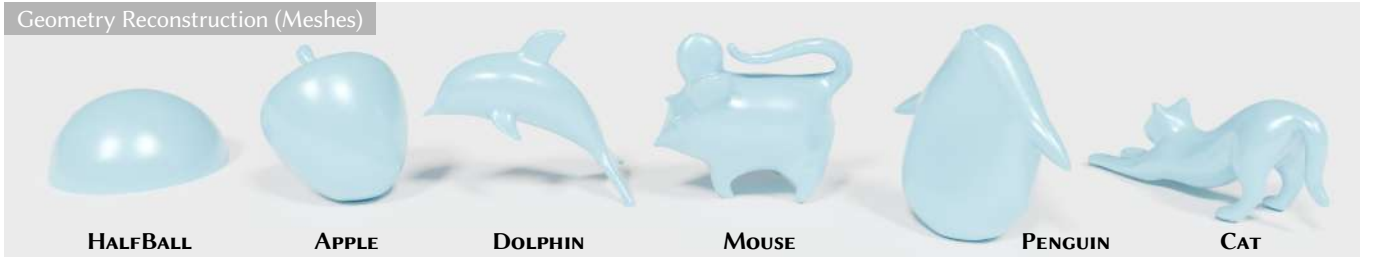


Fig. 9. **Surface mesh reconstruction results of our method on the real-captured and synthetic datasets.**

we present quantitative results measured with three standard metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM) [Wang et al. 2004] and Learned Perceptual Image Patch Similarity (LPIPS) [Zhang et al. 2018]. For the reconstructed normal maps, we use Mean Angular Error in degrees ( $MAE^\circ$ ) to evaluate the normal reconstruction accuracy.

## 4.2 Results and Evaluation

**Qualitative comparison.** In Fig. 8 and Fig. 10, we evaluate our method on the real-captured dataset. And Fig. 11 showcases the evaluation on the synthetic dataset. As seen, our method is capable of reconstructing transparent objects both effectively and efficiently. It is observed that in some cases even NU-NeRF [Sun et al. 2024] has remaining artifacts that our method avoids, such as GLASS and

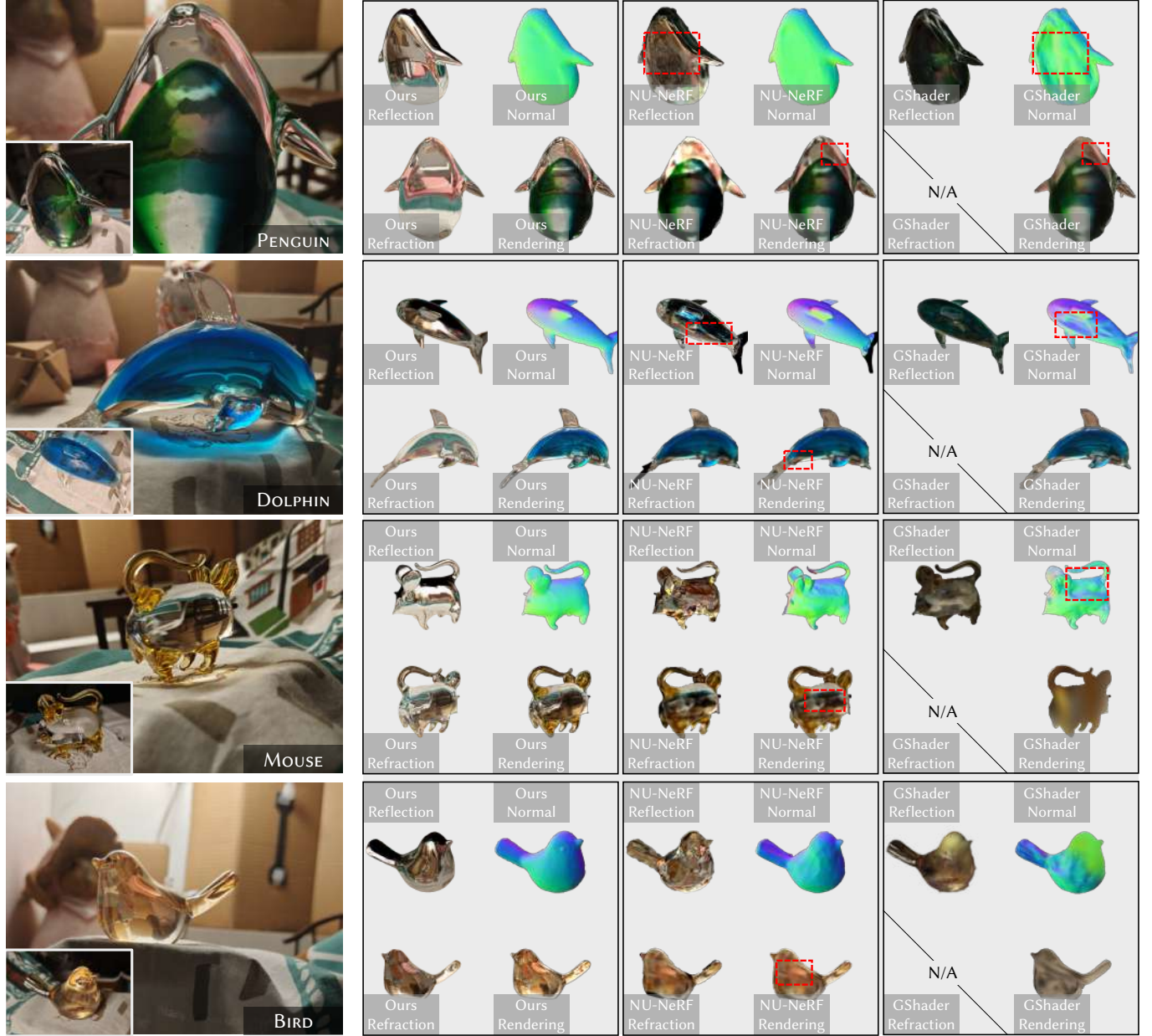


Fig. 10. **Qualitative comparison of novel-view synthesis and inverse rendering results on real-captured scenes with colored transparent objects.** The left images showcase the input images of the real-captured scenes with colored transparent objects. By handling absorption as described in Eqn. (11), we can effectively decouple the refracted light from the objects' inherent texture, whereas NU-NeRF [Sun et al. 2024] can only yield coupled results.

KITTY. And other methods [Bemana et al. 2022; Jiang et al. 2024] cannot faithfully generate normal maps due to the lack of surface representation or refraction modeling. Moreover, our method is adept at capturing the high-frequency details of specular reflection and refraction (e.g., the nearby contents in the inter-reflection - in APPLE - or in the refraction - in MOUSE). NU-NeRF [Sun et al. 2024] tends to generate blurred results and fails to capture the detailed indirect light due to its light representation. It also should be noted that we can handle colored transparent objects and decouple the

refraction term from the inherent color. Previous methods either fail to handle absorption or produce entangled results. We provide the surface mesh reconstruction results of these transparent objects, as shown in Fig. 9. The detailed intermediate results of our method are illustrated in Fig. 12.

*Quantitative comparisons.* Tab. 3 and Tab. 4 show the quantitative comparison of novel view synthesis results on the real-captured datasets. We surpass other methods in most scenes, particularly in

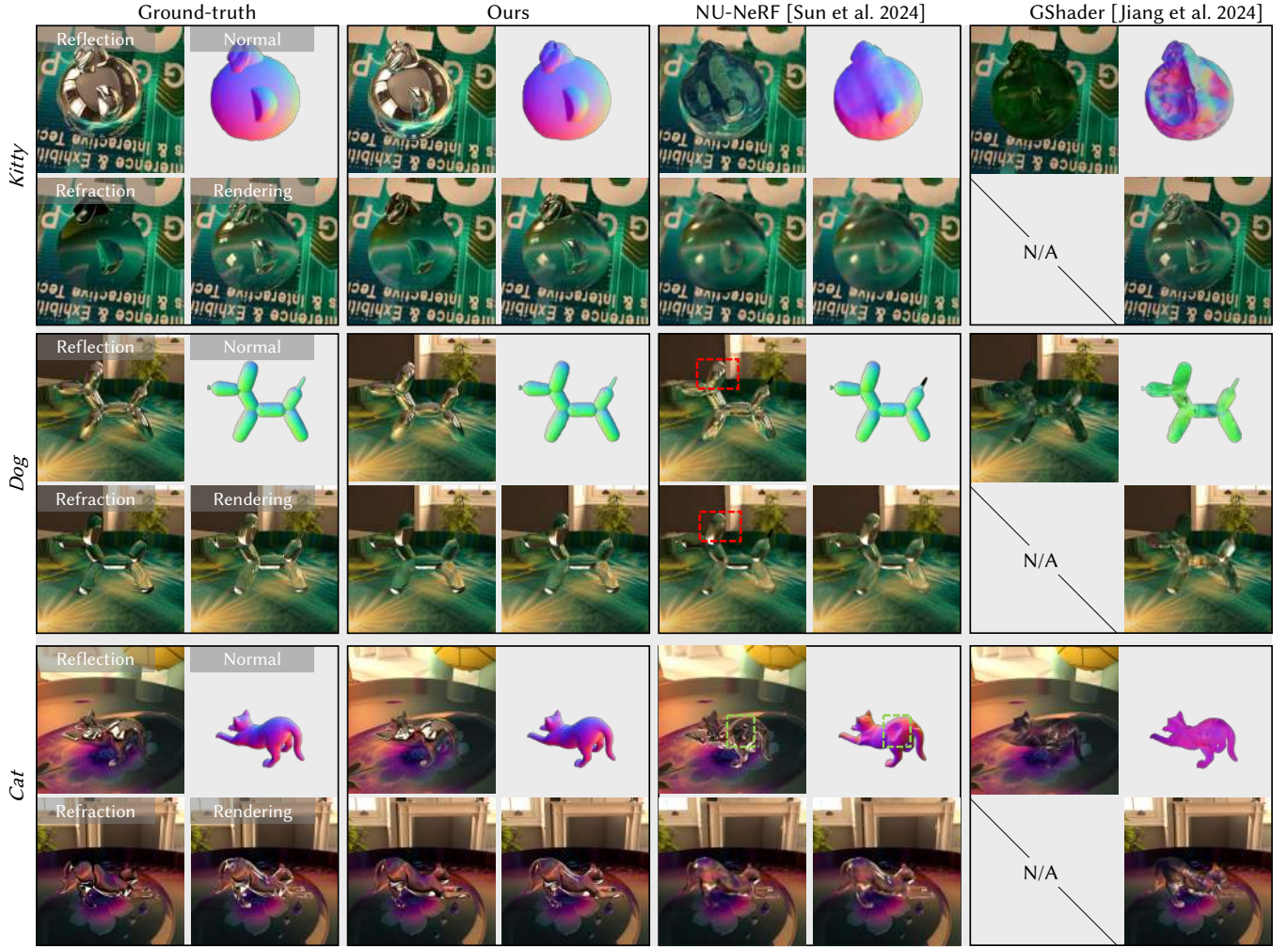


Fig. 11. Qualitative comparison of novel-view synthesis and inverse rendering results on the synthetic dataset.

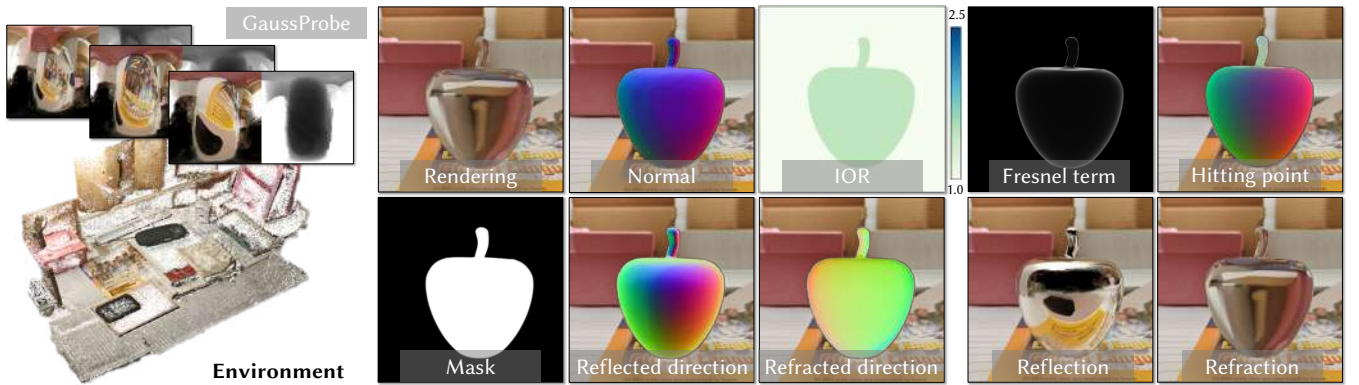


Fig. 12. Detailed intermediate results of our method. Left: the environment and the corresponding GaussProbe. Right: maps of additional parameters.

terms of LPIPS, and achieve the highest average metrics. Since real-captured scenes lack ground-truth geometry, material, and light,

we also evaluate our method on the synthetic dataset, as reported

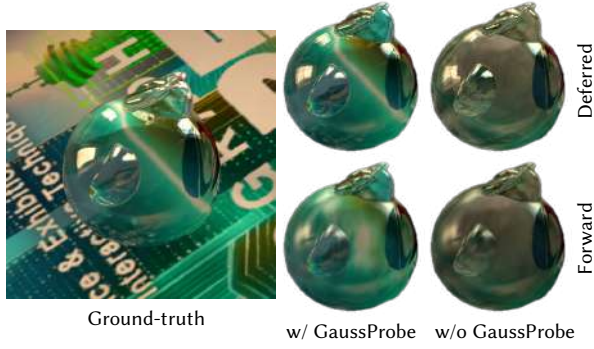


Fig. 13. **Qualitative ablation studies on the KITTY scene.** “Deferred” and “Forward” mean the deferred shading strategy and the forward shading strategy, respectively.

Table 6. **Quantitative ablation studies on the KITTY scene.** “Deferred” and “Forward” mean the deferred shading strategy and the forward shading strategy, respectively.

| Deferred | GaussProbe | PSNR↑        | SSIM↑        | LPIPS↓       |
|----------|------------|--------------|--------------|--------------|
| ✗        | ✗          | 27.05        | 0.958        | 0.044        |
| ✗        | ✓          | 27.89        | 0.963        | 0.040        |
| ✓        | ✗          | 27.13        | 0.958        | 0.043        |
| ✓        | ✓          | <b>28.41</b> | <b>0.970</b> | <b>0.036</b> |

in Tab. 5. We achieve state-of-the-art results in both novel view synthesis and inverse rendering. In terms of novel view synthesis on the synthetic dataset, the relatively high score of GShader [Jiang et al. 2024] can be attributed to its spherical harmonic function that overfits the views, which also prevents it from reconstructing the correct normal.

*Performance analysis.* Generally, our method achieves superior inverse rendering quality compared to Eikonal [Bemana et al. 2022] (22-24 hours) and NU-NeRF [Sun et al. 2024] (8-9 hours), while requiring a comparable training time to GShader [Jiang et al. 2024] (1 hour). We segment the 3D scene and bake GaussProbe within a few minutes. In terms of novel view synthesis and re-rendering performance, the runtime depends on the number of probes, the number of iterations, and the image resolution. For an  $800 \times 800$  image, the time cost is 0.002 seconds for a single probe with 1 iteration and 0.005 seconds for 8 probes with 5 iterations. Our transparent Gaussian primitives, similar to other 3D-GS-based methods [Jiang et al. 2024; Ye et al. 2024], are capable of rendering various attributes maps in real-time for deferred shading. Due to the efficient designs, our method achieves the frame rate of 31-51, which is significantly faster than existing NeRF-based works: Eikonal [Bemana et al. 2022] (0.03 FPS) and NU-NeRF [Sun et al. 2024] (0.016 FPS).

### 4.3 Ablation Studies

In this section, we conduct various ablation studies to validate the impact of key components in our method.

*Ablation studies on inverse rendering.* The key components of our method for inverse rendering of transparent objects are the deferred

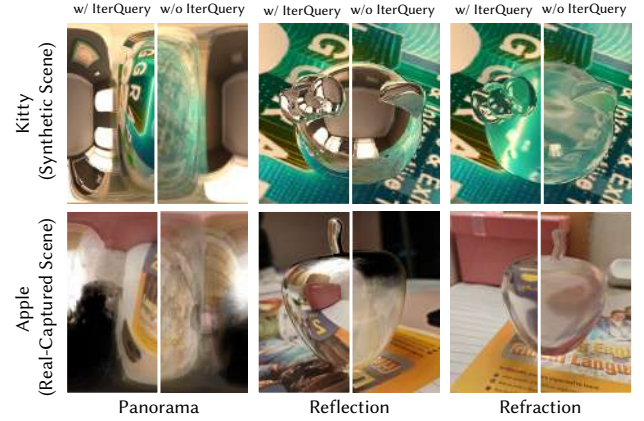


Fig. 14. **The impact of our IterQuery algorithm on rendering panoramas, reflection, and refraction.** Left half of each image: with IterQuery; Right half: without IterQuery. For the reflection, since the window is far enough from KITTY, the parallax effect is small, resulting in only slight visual differences.

refraction strategy and our Gaussian light field probes. Thus, we provide quantitative and qualitative ablation studies under different settings on the KITTY scene, as reported in Tab. 6 and shown in Fig. 13. It can be observed that the forward shading strategy blurs the high-frequency details of refraction and reflection in the rendering results. Without using probes, the overall color of the refracted light from the nearby table deviates significantly from the ground truth, but the high-frequency reflected light of ambient light from an “infinite distance” are preserved. Simultaneously employing deferred shading and Gaussian light field probes effectively combines the benefits of preserving high-frequency details and ensuring accurate query directions. The metrics further tell that our full method achieves the highest quality.

*Ablation studies on the IterQuery algorithm.* Given that the IterQuery algorithm plays a critical role in modeling indirect inter-reflection and refraction, we further conduct ablation studies on the algorithm.

We visualize the rendered panoramas, reflection maps, and refraction maps using Gaussian light field probes in Fig. 14. It can be observed that without IterQuery, both the panoramas and refraction maps are over-blurred due to the parallax introduced by the probes. The reflected light from the window at the top of KITTY is sufficiently far away, making the parallax introduced by the probes negligible. However, the inter-reflected light at the bottom of KITTY is also blurred due to its proximity to the tabletop. On the contrary, our IterQuery algorithm effectively addresses the parallax issue and enhances the details of rendering.

Different probes configurations, such as the count of iterations, the number and the placement of probes, also influence the convergence of the iteration and the rendering results, as illustrated in Fig. 15. When  $K = 1$ , the convergence is highly sensitive to the placement of the probe. The limited receptive field of the single probe makes it difficult to converge to the correct solution. As the



Fig. 15. **The impact of probes configurations on the convergence of the IterQuery algorithm.** The leftmost image illustrates the number of the probes  $K$  and the placement of the probes. In the 3rd iteration of the second row’s reflection, it appears clearer but is incorrect, as only one line of text should be reflected at the bottom (red box).

iterations progress, the rendering results for the cases  $K = 8, 64$  improve steadily, as indicated by the red boxes. Moreover, it can be observed that the case  $K = 64$  converges more rapidly. Theoretically, as  $K$  approaches infinity—where each incoherent ray is rasterized individually during the first iteration—the result converges to the optimal solution immediately. This provides a clear explanation for why larger  $K$  values require fewer iterations for convergence.

#### 4.4 Applications

Efficient Gaussian light fields open the door to many advanced applications, including secondary ray effects like mirrors, refractions, as well as rendering with non-pinhole cameras, as shown in Fig. 16. Please review our supplemental video for more details.

*Re-rendering of reconstructed transparent objects.* We place the transparent object MOUSE reconstructed with our TransparentGS into the scene DRJOHNSON [Hedman et al. 2018] reconstructed with the original 3D-GS [Kerbl et al. 2023] for re-rendering. Thanks to the reconstruction quality of TransparentGS and the query quality of GaussProbe, we can obtain photorealistic results for relighting and material editing. Additionally, our method demonstrates a significant performance advantage over other NeRF-based approaches.

*Secondary ray effects of inserted meshes.* We use traditional triangle meshes (BALL, STANFORDBUNNY, HAPPY, MIRROR, and LUCK, the colored plane), 3D-GS [Kerbl et al. 2023] (DRJOHNSON, PLAYROOM [Hedman et al. 2018], TRUCK [Knapitsch et al. 2017], STUMP, BICYCLE, GARDEN [Barron et al. 2022], and LEGO [Mildenhall et al. 2020]), and transparent Gaussian primitives (MOUSE, APPLE, PENGUIN, DOG, and DOLPHIN) to create a variety of scenes. GaussProbe enable secondary ray effects in these multi-object composition scenes, as shown in Fig. 1 and Fig. 16, which is challenging for other methods.

*Rendering with non-pinhole camera models.* By the way, we can also leverage GaussProbe for rendering with non-pinhole cameras, such as PLAYROOM [Hedman et al. 2018] with fisheye cameras and panoramas.

#### 4.5 Discussions and Limitations

*Robustness of our method to segmentation results.* Since we use semantic segmentation to partition the scene, we validate the impact of segmentation results on the reconstruction of transparent objects, as shown in Fig. 17. It can be observed that although the segmentation results are mostly accurate, there are still imprecise regions, such as the dolphin’s mouth and the mouse’s tail. However,



Fig. 16. **Applications.** Our method supports the rendering and navigation of scenes that integrate triangle meshes, traditional 3D-GS [Kerbl et al. 2023] and transparent Gaussian primitives, as well as non-pinhole cameras.

our method demonstrates sufficient robustness to such inaccuracy through supervision from multiple views.

**Manifold constraint.** For our Gaussian light field probes, a single probe with the depth panorama can represent a two-dimensional manifold embedded in 3D space. The IterQuery algorithm can be viewed as the process of a moving point sliding along the manifold until it coincides with the query ray. However, as illustrated

in Fig. 18, there are cases where noticeable discontinuities occur, violating the manifold constraint. In such cases, it is possible that not all probes converge to the first intersection between the rays and the 3D scene, leading to incorrect results. This also offers an explanation for the failure case  $K = 1$ , as illustrated in Fig. 15. Fortunately, this can be resolved by increasing the number of probes and employing a more optimal placement strategy.

**Complex light paths and ambiguity.** The light paths are crucial for simulating how light behaves in a transparent object. Similar to previous methods [Gao et al. 2023; Li et al. 2023, 2020], we restrict the number of light bounces. Specifically, we assume that a light path consists of exactly two refractions, with at most one total internal reflection, such as HALFBALL in Fig. 8. However, complex light paths with more bounces can introduce ambiguity and singularity, posing challenges for inverse rendering. Therefore, accurately reconstructing complex geometries, such as hollow transparent objects and those with intricate self-occlusion, remains challenging when using consumer-level cameras. Furthermore, although our method supports varying IOR (Fig. 19), highly heterogeneous transparent objects may introduce ambiguity in material estimation. This may be addressed by a generative model, which we leave as future work.

**Invisible environment.** Similar to existing inverse rendering methods based on NeRF [Mildenhall et al. 2020] or 3D-GS [Kerbl et al. 2023] (e.g., Eikonal [Bemana et al. 2022] or GShader [Jiang et al. 2024]), incomplete environment will affect the reconstruction of invisible parts. Therefore, to improve the accuracy of reconstruction, it is better to capture the whole environment.

**Gaussian ray tracing.** We didn't choose 3D Gaussian Ray Tracing (3DGRT) [Moenne-Loccoz et al. 2024] in our pipeline due to the following reasons. Firstly, 3DGRT currently does not support inverse rendering and material decomposition, as claimed in its limitations. Secondly, unlike 3DGRT, our GaussProbe does not need to build a BVH or maintain a sorted buffer, making it more efficient for our task. Moreover, GaussProbe can be directly obtained from scenes trained with the original 3DGS, whereas 3DGRT requires fine-tuning. That said, it would be an interesting future work to extend 3DGRT to reconstruct transparent objects.

**Caustics rendering.** Since our task is inverse rendering, we focus more on the influence of the surrounding on the object's appearance than the influence of the object on its surrounding (e.g., caustics). Currently, 3DGS-based methods fail to handle caustics rendering. It would be an interesting future work.

**Video.** We provide a video in the supplementary material. Some visual artifacts in the video primarily arise from the aliasing caused by sampling, the accuracy of the marching cube algorithm, and the moving object bounding boxes. These issues can potentially be mitigated by increasing the samples per pixel, and increasing the coverage of the probes to fully encompass the area within the dynamic object's movement range, etc.

## 5 CONCLUSION

In conclusion, we propose TransparentGS, an innovative and efficient inverse rendering framework for transparent objects based on

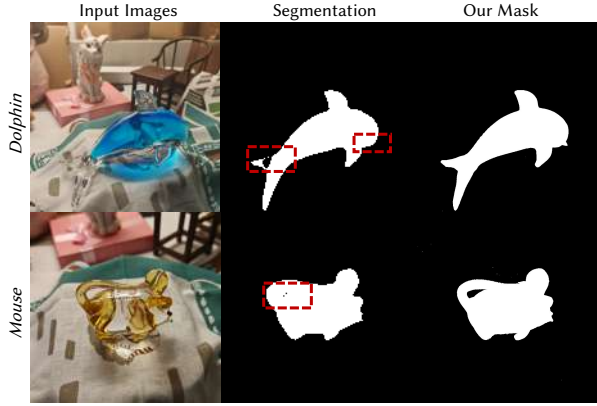


Fig. 17. **Robustness of our method to segmentation results.** The red boxes highlight the regions with imprecise segmentation results.

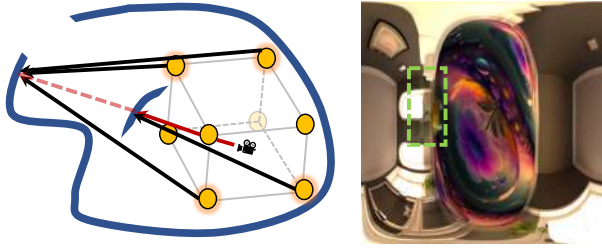


Fig. 18. **Failure cases for the IterQuery algorithm.** The green box highlights the regions where our IterQuery algorithm fails when there are non-negligible singularities that clearly violate the manifold constraint.

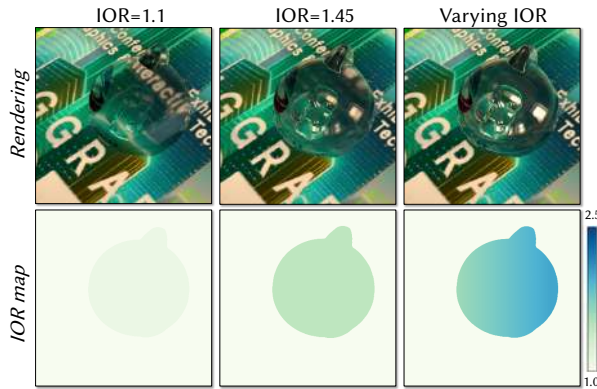


Fig. 19. **Transparent objects with different IORs.** Higher index of refraction values are depicted in blue, and lower values are depicted in green.

3D-GS. By introducing the transparent Gaussian primitives, which incorporate material attributes alongside positions and shapes, our method enables physically-based rendering of transparent materials. To preserve refractive details, we leverage a deferred refraction strategy, to effectively capture specular refraction. Addressing the challenges posed by the high computational cost of ray tracing and the limitations of rasterization in handling incoherent secondary

rays, we employ the Gaussian light field probes (GaussProbe) to represent nearby and distant light variations, accompanied by an IterQuery algorithm that mitigates the parallax artifacts in probe query. The GaussProbe-based pipeline allows us to reconstruct any transparent object within one hour and facilitates realistic rendering. It opens the door to complex secondary ray effects in interplay of the multi-object composition scene. Experimental results confirm the effectiveness and efficiency of our framework, highlighting its potential for advancing transparent objects.

## ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for their valuable feedback. We also thank the authors of Zhou et al. [2024] for generously providing the LaTeX template of the pseudocode. This work was supported by the National Natural Science Foundation of China (No. 61972194 and No. 62032011) and the Natural Science Foundation of Jiangsu Province (No. BK20211147).

## REFERENCES

- Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. 2022. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 5470–5479.
- Frederick O Bartell, Eustace L Dereniak, and William L Wolfe. 1981. The theory and measurement of bidirectional reflectance distribution function (BRDF) and bidirectional transmittance distribution function (BTDF). In *Radiation scattering in optical systems*, Vol. 257. SPIE, 154–160.
- Mojtaba Bermana, Karol Myszkowski, Jeppe Revall Frisvad, Hans-Peter Seidel, and Tobias Ritschel. 2022. Eikonal fields for refractive novel-view synthesis. In *ACM SIGGRAPH 2022 Conference Proceedings*. 1–9.
- Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. 2022. Tensorf: Tensorial radiance fields. In *European conference on computer vision*. Springer, 333–350.
- Robert L Cook and Kenneth E Torrance. 1981. A reflectance model for computer graphics. *ACM Siggraph Computer Graphics* 15, 3 (1981), 307–316.
- Michael Deering, Stephanie Winner, Bic Schediwy, Chris Duffy, and Neil Hunt. 1988. The triangle processor and normal vector shader: a VLSI system for high performance graphics. *Acm siggraph computer graphics* 22, 4 (1988), 21–30.
- Jan-Niklas Dihlmann, Arjun Majumdar, Andreas Engelhardt, Raphael Braun, and Hendrik Lensch. 2024. Subsurface Scattering for Gaussian Splatting. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Fangzhou Gao, Lianghao Zhang, Li Wang, Jiamin Cheng, and Jiawan Zhang. 2023. Transparent Object Reconstruction via Implicit Differentiable Refraction Rendering. In *SIGGRAPH Asia 2023 Conference Papers*. 1–11.
- Jian Gao, Chun Gu, Youtian Lin, Zhihao Li, Hao Zhu, Xun Cao, Li Zhang, and Yao Yao. 2024. Relightable 3D Gaussians: Realistic Point Cloud Relighting with BRDF Decomposition and Ray Tracing. In *European Conference on Computer Vision*. Springer, 73–89.
- Wenhao Ge, Tao Hu, Haoyu Zhao, Shu Liu, and Ying-Cong Chen. 2023. Ref-neus: Ambiguity-reduced neural implicit surface learning for multi-view reconstruction with reflection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4251–4260.
- Antoine Guédon and Vincent Lepetit. 2024. Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5354–5363.
- Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. 2018. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (ToG)* 37, 6 (2018), 1–15.
- Leif Van Holland, Ruben Bliersbach, Jan U Müller, Patrick Stotko, and Reinhard Klein. 2024. TraM-NeRF: Tracing Mirror and Near-Perfect Specular Reflections Through Neural Radiance Fields. In *Computer Graphics Forum*, Vol. 43. Wiley Online Library, e15163.
- Letian Huang, Jiayang Bai, Jie Guo, Yuanqi Li, and Yanwen Guo. 2024. On the Error Analysis of 3D Gaussian Splatting and an Optimal Projection Strategy. In *European Conference on Computer Vision*. Springer, 247–263.
- Cong Phuoc Huynh, Antonio Robles-Kelly, and Edwin Hancock. 2010. Shape and refractive index recovery from single-view polarisation images. In *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. IEEE, 1229–1236.
- Ivo Ihrke, Kiriakos N Kutulakos, Hendrik PA Lensch, Marcus Magnor, and Wolfgang Heidrich. 2010. Transparent and specular object reconstruction. In *Computer graphics*

- forum, Vol. 29. Wiley Online Library, 2400–2426.
- Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin Ma. 2024. Gaussianshader: 3d gaussian splatting with shading functions for reflective surfaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5322–5332.
- Haian Jin, Isabella Liu, Peijia Xu, Xiaoshuai Zhang, Songfang Han, Sai Bi, Xiaowei Zhou, Zexiang Xu, and Hao Su. 2023. Tensor: Tensorial inverse rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 165–174.
- James T Kajiya. 1986. The rendering equation. *ACM SIGGRAPH Computer Graphics* 20, 4 (1986), 143–150.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (2023).
- Erum Arif Khan, Erik Reinhard, Roland W Fleming, and Heinrich H Bühlhoff. 2006. Image-based material editing. *ACM Transactions on Graphics (TOG)* 25, 3 (2006), 654–663.
- Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4015–4026.
- Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. 2017. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics (TOG)* 36, 4 (2017), 1–13.
- Kiriakos N Kutulakos and Eron Steger. 2008. A theory of refractive and specular 3D shape by light-path triangulation. *International Journal of Computer Vision* 76 (2008), 13–29.
- Shuichang Lai, Letian Huang, Jie Guo, Kai Cheng, Bowen Pan, Xiaoxiao Long, Jiangjing Lyu, Chengfei Lv, and Yanwen Guo. 2025. GlossyGS: Inverse Rendering of Glossy Objects With 3D Gaussian Splatting. *IEEE Transactions on Visualization and Computer Graphics* (2025), 1–14.
- Jia Li, Lu Wang, Lei Zhang, and Beibei Wang. 2024. Tensosdf: Roughness-aware tensorial representation for robust geometry and material reconstruction. *ACM Transactions on Graphics (TOG)* 43, 4 (2024), 1–13.
- Zongcheng Li, Xiaoxiao Long, Yusen Wang, Tuo Cao, Wenping Wang, Fei Luo, and Chunxia Xiao. 2023. NeTO: neural reconstruction of transparent objects with self-occlusion aware refraction-tracing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 18547–18557.
- Zhengqin Li, Yu-Ying Yeh, and Manmohan Chandraker. 2020. Through the looking glass: Neural 3d reconstruction of transparent shapes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1262–1271.
- Ruofan Liang, Huiting Chen, Chunlin Li, Fan Chen, Selvakumar Panneer, and Nandita Vijaykumar. 2023. Envidr: Implicit differentiable renderer with neural environment lighting. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 79–89.
- Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. 2024. Gs-ir: 3d gaussian splatting for inverse rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 21644–21653.
- Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, et al. 2024. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*. Springer, 38–55.
- Yuan Liu, Peng Wang, Cheng Lin, Xiaoxiao Long, Jiepeng Wang, Lingjie Liu, Taku Komura, and Wenping Wang. 2023. Nero: Neural geometry and brdf reconstruction of reflective objects from multiview images. *ACM Transactions on Graphics (TOG)* 42, 4 (2023), 1–22.
- William E Lorensen and Harvey E Cline. 1987. Marching cubes: A high resolution 3D surface construction algorithm. *ACM SIGGRAPH Computer Graphics* 21, 4 (1987), 163–169.
- Jiahui Lyu, Bojian Wu, Dani Lischinski, Daniel Cohen-Or, and Hui Huang. 2020. Differentiable refraction-tracing for mesh reconstruction of transparent objects. *ACM Transactions on Graphics (TOG)* 39, 6 (2020), 1–13.
- Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *European Conference on Computer Vision*. Springer, 405–421.
- Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 2024. 3D Gaussian Ray Tracing: Fast Tracing of Particle Scenes. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–19.
- Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)* 41, 4 (2022), 1–15.
- Yiming Qian, Minglun Gong, and Yee Hong Yang. 2016. 3d reconstruction of transparent objects with position-normal consistency. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4369–4377.
- Nikhila Ravi, Valentín Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. 2024. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714* (2024).
- Christophe Schlick. 1994. An inexpensive BRDF model for physically-based rendering. In *Computer graphics forum*, Vol. 13. Wiley Online Library, 233–246.
- Mingqi Shao, Chongkun Xia, Dongxu Duan, and Xueqian Wang. 2024. Polarimetric inverse rendering for transparent shapes reconstruction. *IEEE Transactions on Multimedia* (2024).
- Jonathan Dyssel Stets, Alessandro Dal Corso, Jannik Boll Nielsen, Rasmus Ahrenkiel Lyngby, Sebastian Hoppe Nesgaard Jensen, Jakob Wilm, Mads Brix Doest, Carsten Gundlach, Eythor Runar Eiriksson, Knut Conradsen, et al. 2017. Scene reassembly after multimodal digitization and pipeline evaluation using photorealistic rendering. *Applied optics* 56, 27 (2017), 7679–7690.
- Jia-Mu Sun, Tong Wu, Ling-Qi Yan, and Lin Gao. 2024. NU-NeRF: Neural Reconstruction of Nested Transparent Objects with Uncontrolled Capture Environment. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–14.
- Jinguang Tong, Sundaram Muthu, Fahira Afzal Maken, Chuong Nguyen, and Hongdong Li. 2023. Seeing through the glass: Neural 3d reconstruction of object inside a transparent container. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12555–12564.
- Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T Barron, and Pratul P Srinivasan. 2022. Ref-nerf: Structured view-dependent appearance for neural radiance fields. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 5481–5490.
- Dongqing Wang, Tong Zhang, and Sabine Süsstrunk. 2023. Nemto: Neural environment matting for novel view and relighting synthesis of transparent objects. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 317–327.
- Peng Wang, Lingjie Liu, Yuan Liu, Christian Theobalt, Taku Komura, and Wenping Wang. 2021. NeuS: Learning Neural Implicit Surfaces by Volume Rendering for Multi-view Reconstruction. *Advances in Neural Information Processing Systems* 34 (2021), 27171–27183.
- Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing* 13, 4 (2004), 600–612.
- Gordon Wetzstein, David Roodnick, Wolfgang Heidrich, and Ramesh Raskar. 2011. Refractive shape from light field distortion. In *2011 International Conference on Computer Vision*. IEEE, 1180–1186.
- Bojian Wu, Yang Zhou, Yiming Qian, Minglun Cong, and Hui Huang. 2018. Full 3D reconstruction of transparent objects. *ACM Transactions on Graphics (TOG)* 37, 4 (2018), 1–11.
- Tong Wu, Jia-Mu Sun, Yu-Kun Lai, Yuewen Ma, Leif Kobbelt, and Lin Gao. 2024a. DeferredGS: Decoupled and Editable Gaussian Splatting with Deferred Shading. *arXiv preprint arXiv:2404.09412* (2024).
- Tong Wu, Yu-Jie Yuan, Ling-Xiao Zhang, Jie Yang, Yan-Pei Cao, Ling-Qi Yan, and Lin Gao. 2024b. Recent advances in 3d gaussian splatting. *Computational Visual Media* 10, 4 (2024), 613–642.
- Jiamin Xu, Zihan Zhu, Hujun Bao, and Weiwei Xu. 2022. A hybrid mesh-neural representation for 3D transparent object reconstruction. *ACM Trans. Graph* 1, 1 (2022).
- Keyang Ye, Qiming Hou, and Kun Zhou. 2024. 3d gaussian splatting with deferred reflection. In *ACM SIGGRAPH 2024 Conference Papers*. 1–10.
- Zinuo You, Andreas Geiger, and Anpei Chen. 2024. NeLF-Pro: Neural Light Field Probes for Multi-Scale Novel View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 19833–19843.
- Mulin Yu, Tao Lu, Linning Xu, Lihan Jiang, Yuanbo Xiangli, and Bo Dai. 2024. GSDF: 3DGS Meets SDF for Improved Neural Rendering and Reconstruction. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Yifan Zhan, Shohei Nobuhara, Ko Nishino, and Yinqiang Zheng. 2023. Nerfrac: Neural radiance fields through refractive surface. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 18402–18412.
- Jingyang Zhang, Yao Yao, Shiwei Li, Jingbo Liu, Tian Fang, David McKinnon, Yanghai Tsin, and Long Quan. 2023. Neilf++: Inter-reflectable light fields for geometry and material estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3601–3610.
- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. 2018. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 586–595.
- Yang Zhou, Songyin Wu, and Ling-Qi Yan. 2024. Unified Gaussian Primitives for Scene Representation and Rendering. *arXiv preprint arXiv:2406.09733* (2024).
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. 2002. EWA splatting. *IEEE Transactions on Visualization and Computer Graphics* 8, 3 (2002), 223–238.